《数据库系统》
期末复习回顾

2022年12月

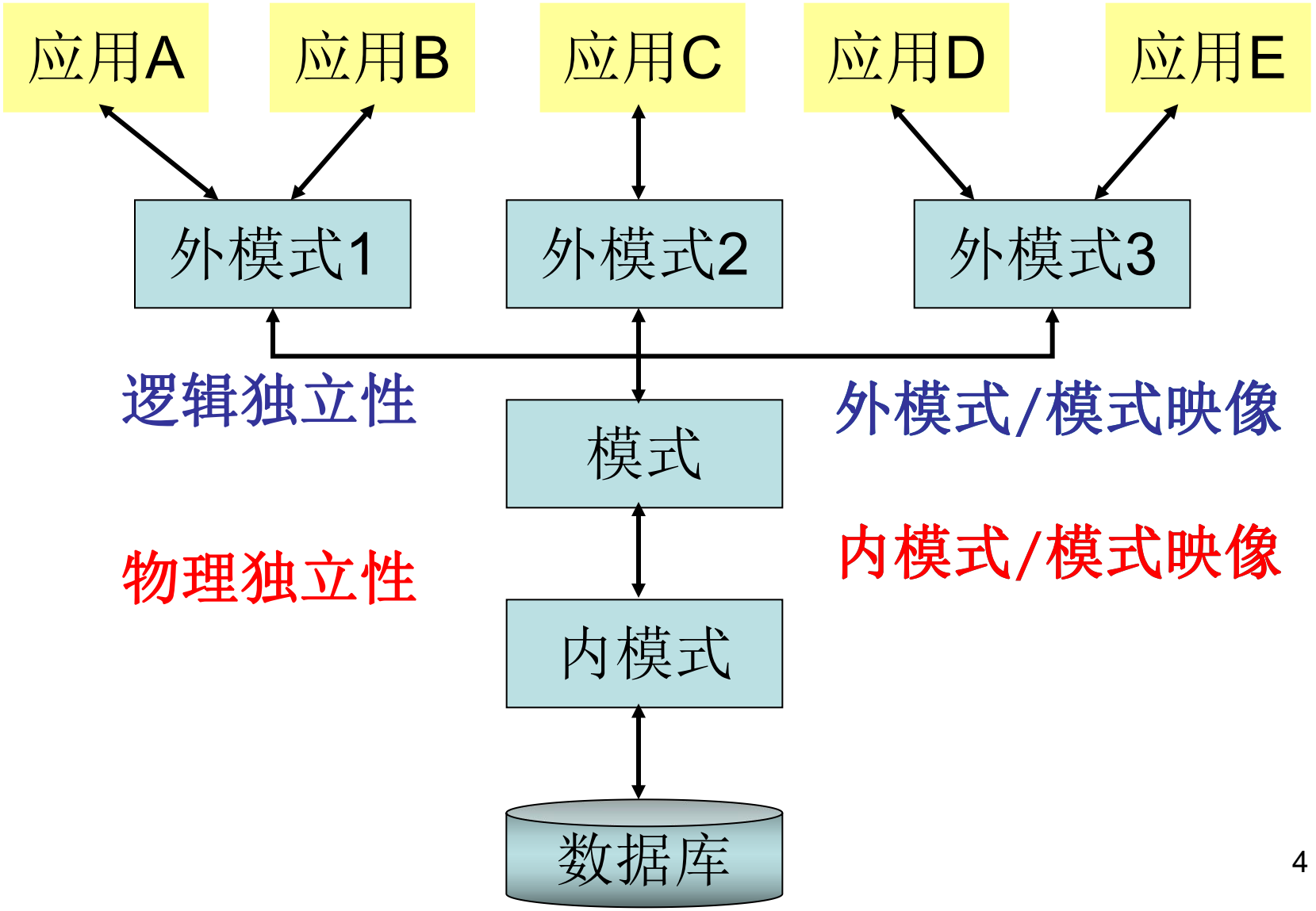
题型

一、选择题(共10题，每题2分，共20分)
二、填空题(共10空，每空1分，共10分)
三、关系代数计算题（共5题，每题2分，共10分）
四、SQL计算题(共10题，每题3分，共30分)
五、简答题(共5题，每题4分，共20分)
六、综合分析题(共10分)

Chapter 1 绪论

- 数据库定义: Page 4
- 数据模型三要素：数据结构、数据操作、完整性约束
- 数据库领域常用的逻辑模型
- 概念模型 (E-R图)
- 什么是数据库系统三级模式结构？优点？
  - 三层结构、两个映像、物理独立性、逻辑独立性

数据库系统三级模式结构



Chapter 2 关系数据库

- 域，笛卡尔积，关系，元组，属性
- 候选码，主码，外部码，主属性，非主属性，全码

关系代数

- 集合运算符
- 专门的关系运算符
- 算术比较符
- 逻辑运算符

会写8种关系代数符号的定义
会写关系代数查询

第三章 关系数据库标准语言SQL

3.1 SQL概述

3.2 学生—课程数据库

3.3 数据定义

3.4 数据查询

3.5 数据更新

3.6 空值的处理

3.7 视图

3.8 小结

SQL的特点

①综合统一

②高度非过程化

③面向集合的操作方式

④以同一种语法结构提供多种使用方式：交互式、嵌入式

⑤语言简洁，易学易用

3.3 数据定义

模式定义、表定义、视图定义、索引定义

操作对象	操作方式		
	创建	删除	修改
模式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视图	CREATE VIEW	DROP VIEW	
索引	CREATE INDEX	DROP INDEX	ALTER INDEX

数据查询语句格式

SELECT [ALL|DISTINCT]
<目标列表达式>[，<目标列表达式>] …
FROM <表名或视图名>[，<表名或视图名>] …
[WHERE <条件表达式>]
[GROUP BY <列名1>
[HAVING <条件表达式>]]
[ORDER BY <列名2> [ASC|DESC]];

注意不写中括号[ ]

3.4 数据查询

3.4.1 单表查询

3.4.2 连接查询

3.4.3 嵌套查询

3.4.4 集合查询

3.4.5 基于派生表的查询

3.4.6 Select语句的一般格式

3.4.1 单表查询

仅涉及一个表的查询

①选择表中的若干列

②选择表中的若干元组

③ORDER BY子句

④聚集函数 (aggregation function)

⑤GROUP BY子句

GROUP BY子句

- 将查询结果按某一列或多列的值分组，值相等的为一组
- 对查询结果分组的目的是为了细化聚集函数的作用对象

① 未对查询结果分组，聚集函数将作用于整个查询结果

② 对查询结果分组后，聚集函数将分别作用于每个组

③ 作用对象是查询的中间结果表

HAVING短语与WHERE子句的区别

①作用对象不同

②WHERE子句作用于基本表或视图，从中选择满足条件的元组

③HAVING短语作用于组，从中选择满足条件的组

3.4.2 连接查询

连接查询：同时涉及多个表的查询

连接条件或连接谓词：连接两个表的条件

一般格式

① [<表名1>.]<列名1> <比较运算符> [<表名2>.]<列名2>

② [<表名1>.]<列名1> BETWEEN [<表名2>.]<列名2> AND [<表名3>.]<列名3>

连接查询的种类

一、等值与非等值连接查询

二、自身连接

三、外连接

四、多表连接

嵌套查询

- 一个SELECT-FROM-WHERE语句称为一个查询块
- 将一个查询块嵌套在另一个查询块的WHERE子句或HAVING短语的条件中的查询称为嵌套查询 (Nested Query)

嵌套查询

```
SELECT Sname /*外层查询或父查询*/
FROM Student
WHERE Sno IN
    (SELECT Sno /*内层查询或子查询*/
    FROM SC
    WHERE Cno= '2' );
```

嵌套查询

- 子查询的限制

  ① 不能使用ORDER BY子句

  ② ORDER BY子句只对最终查询结果排序

- 层层嵌套方式反映了SQL语言的结构化特点

嵌套查询求解方法

- 不相关子查询：子查询的查询条件不依赖于父查询。由里向外逐层处理。即每个子查询在上一级查询处理之前求解，子查询的结果用于建立其父查询的查找条件。

嵌套查询求解方法

相关子查询：子查询的查询条件依赖于父查询

① 首先取外层查询表第一个元组，根据它与内层查询相关的属性值处理内层查询，若WHERE子句返回值为真，则取此元组放入结果表

② 然后再取外层表的下一个元组

③ 重复这一过程，直至外层表全部检查完为止

3.5 数据更新

3.5.1 插入数据

3.5.2 修改数据

3.5.3 删除数据

3.5.1 插入数据

两种插入数据方式

①插入元组

②插入子查询结果：可以一次插入多个元组

一、插入元组

- 语句格式
- INSERT INTO <表名> [(<属性列1>[, <属性列2 >…)] VALUES (<常量1> [, <常量2>] …)
- 功能：将新元组插入指定表中

3.5.2.修改数据

- 语句格式

  UPDATE <表名>

  SET <列名>=<表达式>[，<列名>=<表达式>]...

  [WHERE <条件>];

- SET子句指定修改方式，要修改的列，修改后取值
- WHERE子句指定要修改的元组。缺省表示要修改表中的所有元组

修改数据

三种修改方式

①修改某一个元组的值

②修改多个元组的值

③带子查询的修改语句

3.5.3 删除数据

- 语句格式

  DELETE

  FROM <表名>

  [WHERE <条件>];

- 功能：删除指定表中满足WHERE子句条件的元组
- WHERE子句指定要删除的元组。缺省表示要删除表中的全部元组，表的定义仍在字典中

删除数据

三种删除方式

①删除某一个元组的值

②删除多个元组的值

③带子查询的删除语句

- 视图
  - –视图是从一个或几个基本表导出的表
  - –它与基本表不同，是一个虚表
- 视图的作用
- 视图是否都可以更新？
- 行列子集视图

Chapter 4 数据库安全性

数据库安全性控制常用方法和技术

1. 用户身份鉴别
2. 存取控制
   - 主体、客体定义
   - 自主存取控制方法定义
   - 强制存取控制方法定义
3. 视图机制
4. 审计数据加密
5. 推理控制、隐蔽信道、数据隐私保护

4.2.3 自主存取控制方法 (DAC)

- 通过SQL 的GRANT 语句和REVOKE 语句实现用户权限组成
  - –数据对象
  - –操作类型
- 定义用户存取权限：定义用户可以在哪些数据库对象上进行哪些类型操作
- 定义存取权限称为授权(Authorization)

4.2.4 授权：授予与收回

- GRANT语句的一般格式

  GRANT <权限>[,<权限>]...

  [ON <对象类型> <对象名>]

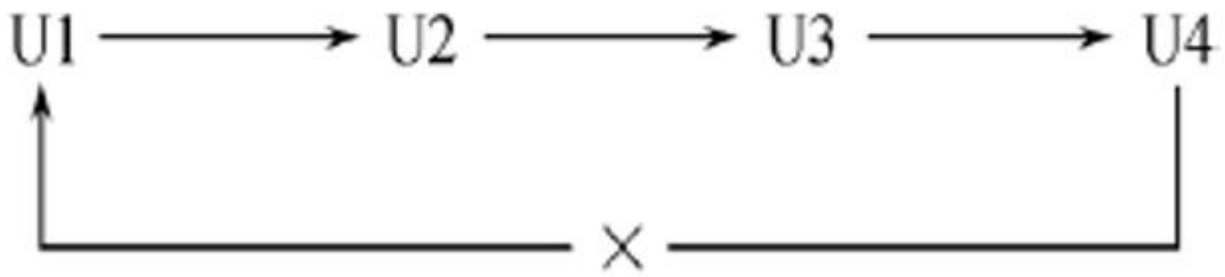
  TO <用户>[,<用户>]...

  [WITH GRANT OPTION];

- 语义：将对指定操作对象的指定操作权限授予指定的用户

WITH GRANT OPTION子句

- 指定：可以再授予
- 没有指定：不能传播
- 不允许循环授权



例5. 把对表SC的INSERT权限授予U5用户，并允许他再将此权限授予其他用户

```
GRANT INSERT
ON TABLE SC
TO U5
WITH GRANT OPTION;
```

Chapter 5 数据库完整性

- 什么是数据库安全性？什么是完整性?
- 数据库完整性和安全性的区别和联系?
- 三类完整性约束

① 实体完整性

② 参照完整性

③ 用户定义的完整性

为维护数据库的完整性，DBMS必须能够

- 提供定义完整性约束条件的机制
- 提供完整性检查的方法
- 违约处理

外码 (Foreign Key)

- 设*F*是基本关系*R*的一个或一组属性，但不是关系*R*的码。如果*F*与基本关系*S*的主码K_s相对应，则称*F*是基本关系*R*的外码
- 基本关系*R* 称为参照关系 (Referencing Relation)
- 基本关系*S* 称为被参照关系 (Referenced Relation) 或目标关系 (Target Relation)

可能破坏参照完整性的情况及违约处理

被参照表（例如Student）	参照表（例如SC）	违约处理
可能破坏参照完整性 ←	插入元组	拒绝
可能破坏参照完整性 ←	修改外码值	拒绝
删除元组 →	可能破坏参照完整性	拒绝/级连删除/设置为空值
修改主码值 →	可能破坏参照完整性	拒绝/级连修改/设置为空值

定义触发器的语法说明

- 创建者：表的拥有者
- 触发器名
- 表名：触发器的目标表
- 触发事件：INSERT、DELETE、UPDATE
- 触发器类型
  - 行级触发器 (FOR EACH ROW)
  - 语句级触发器 (FOR EACH STATEMENT)
- 触发条件
  - 触发条件为真，or 省略WHEN触发条件
- 触发动作体
  - 触发动作体可以是一个匿名PL/SQL过程块
  - 也可以是对已创建存储过程的调用

Chapter 6 关系数据理论

- 关系数据库逻辑设计
  - –针对一个具体问题，如何构造一个适合于它的数据模式（构造几个关系模式？每个关系由哪些属性组成？）
- 数据库逻辑设计的有力工具–关系数据库的规范化理论

什么是数据依赖？

- 一个关系内部属性与属性之间的一种约束关系
- 通过属性间值的相等与否体现出来的数据间相关联系
- 现实世界属性间相互联系的抽象
- 数据内在的性质
- 语义的体现

数据依赖的类型

① 函数依赖 (Functional Dependency，简记为FD)

② 多值依赖 (Multi-valued Dependency，简记为MVD)

关系模式Student<U, F>中存在的问题

Sno	Sdept	Mname	Cno	Grade
S1	CS	张明	C1	95
S2	CS	张明	C1	90
S3	CS	张明	C1	88
S4	CS	张明	C1	70
S5	CS	张明	C1	78
…	…	…		

- 数据冗余太大
- 更新异常
- 插入异常
- 删除异常

- 结论
  - Student关系模式不是一个好的模式
- “好”的模式
  - 不会发生插入异常、删除异常、更新异常，
  - 数据冗余应尽可能少
- 原因：由存在于模式中的某些数据依赖引起
- 解决方法：通过模式分解来消除其中不合适的数据依赖

- 函数依赖
- 非平凡的函数依赖 vs. 平凡的函数依赖
- 完全函数依赖 vs. 部分函数依赖
- 传递依赖

一、函数依赖

定义**6.1** 设*R*(*U*)是属性集*U*上的关系模式。*X*, *Y*是*U*的子集。若对于*R*(*U*)的任意一个可能的关系*r*，*r*中不可能存在两个元组在*X*上的属性值相等，而在*Y*上的属性值不等，则称“*X*函数确定*Y*”或“*Y*函数依赖于*X*”，记作X→Y。

二、非平凡的函数依赖 vs. 平凡的函数依赖

- 在关系模式$R(U)$中，对于U的子集X和Y，如果$X \rightarrow Y$，但$Y \not\subseteq X$，则称$X \rightarrow Y$是非平凡的函数依赖。
- 若$X \rightarrow Y$，但$Y \subseteq X$，则称$X \rightarrow Y$是平凡的函数依赖。

三、完全函数依赖与部分函数依赖

定义6.2 在关系模式$R(U)$中，如果$X \rightarrow Y$，并且对于X的任何一个真子集X'，都有$X' \not\rightarrow Y$，则称Y对X完全函数依赖，记作$X \xrightarrow{F} Y$。

若$X \rightarrow Y$，但Y不完全函数依赖于X，则称Y对X部分函数依赖，记作$X \xrightarrow{P} Y$。

四、传递函数依赖

定义6.3 在关系模式$R(U)$中，如果$X \rightarrow Y$, $(Y \subsetneq X)$， $Y \not\rightarrow X$, $Y \rightarrow Z$，$Z \not\subseteq Y$，则称Z对X传递函数依赖 (transitive functional dependency)。

- 注: 如果$Y \rightarrow X$， 即$X \longleftrightarrow Y$，则Z直接依赖于X
- 例: 在关系Student(Sno, Sdept, Mname)中
  Sno → Sdept，Sdept → Mname，
  Mname传递函数依赖于Sno

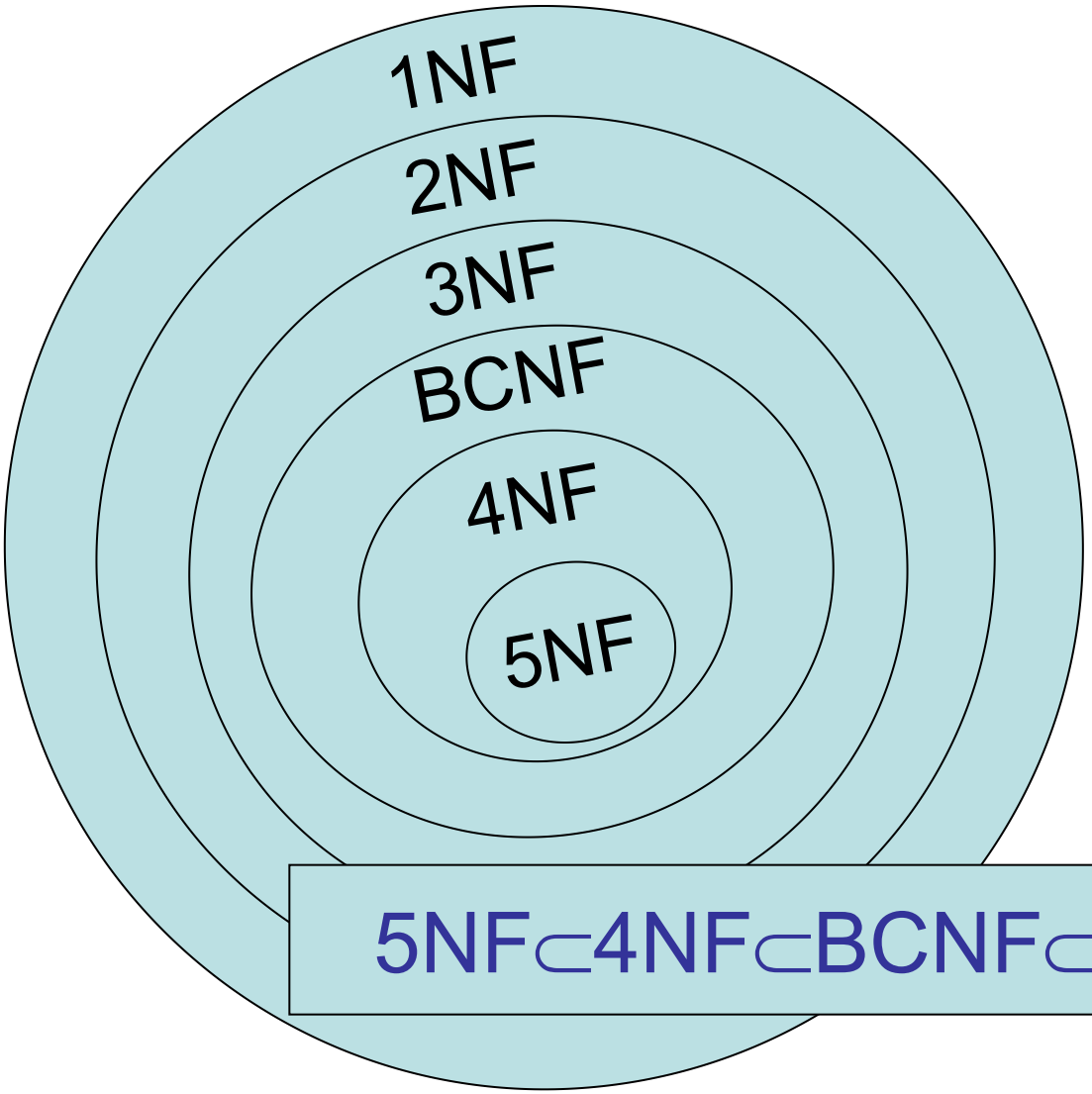
6.2.2 码

- 定义6.4 设*K*为*R*<*U*, *F*>中的属性或属性组合。若 $K \xrightarrow{F} U$，则*K*为*R*的侯选码 (Candidate Key)
- 若候选码多于一个，则选定其中的一个为主码 (Primary Key)

6.2.3 范式

- 范式是符合某一种级别的关系模式的集合。
- 关系数据库中的关系必须满足一定的要求。
- 不同范式满足不同程度要求。

各种范式之间的联系



一个低一级范式的关系模式，通过模式分解可以转换为若干个高一级范式的关系模式的集合，这种过程就叫规范化(normalization)

$5NF \subset 4NF \subset BCNF \subset 3NF \subset 2NF \subset 1NF$

1NF

- 如果一个关系模式*R*的所有属性都是不可分的基本数据项，则 $R \in 1NF$
- 第一范式是对关系模式最起码的要求。不满足第一范式的数据库模式不能称为关系数据库。
- 但是满足第一范式的关系模式并不一定是一个好的关系模式。

2NF

- 定义6.6 若R∈1NF，且每一个非主属性完全函数依赖于码，则R∈2NF。

3NF

- 定义6.7 关系模式$R<U, F>$ 中若不存在这样的码X，属性组Y及非主属性Z $(Z \notin Y)$，使得$X \rightarrow Y$，$Y \rightarrow Z$成立，$Y \not\rightarrow X$，则称$R<U, F> \in 3NF$。
- 若$R \in 3NF$，则每一个非主属性既不是部分依赖于码也不传递依赖于码。

6.2.6 BCNF

- 定义6.8 关系模式R<U, F>∈1NF，若X→Y且Y⊈X时X必含有码，则R<U, F>∈BCNF
- 等价于：每一个决定因素都包含码

多值依赖

- 多值依赖定义
- 多值依赖的性质
- 多值依赖和函数依赖的区别

多值依赖

定义6.9 设$R(U)$是属性集U上的一个关系模式。X, Y, Z是U的子集，并且Z=U-X-Y。关系模式$R(U)$中多值依赖 $X \rightarrow\rightarrow Y$成立，当且仅当对$R(U)$的任一关系$r$，给定一对$(x, z)$值，有一组$Y$的值，这组值仅仅决定于$x$值而与$z$值无关。

多值依赖的性质

① 多值依赖具有对称性

若X→→Y，则X→→Z，其中Z＝U－X－Y

② 多值依赖具有传递性

若X→→Y，Y→→Z， 则X→→Z –Y

③ 函数依赖是多值依赖的特殊情况，若X→Y，则X→→Y

④ 若X→→Y，X→→Z，则X→→YZ

⑤ 若X→→Y，X→→Z，则X→→Y∩Z

⑥ 若X→→Y，X→→Z，则X→→Y-Z，X→→Z-Y

多值依赖与函数依赖的区别

- 多值依赖的有效性与属性集的范围有关
  - ① 若X→→Y在U上成立则在W(XY⊆W⊆U)上一定成立
  - ② 反之不然，若X→→Y在W(W⊂U)上成立，在U上并不一定成立
- 若函数依赖X→Y在R(U)上成立，则对于任何Y′⊂Y均有X→Y′ 成立。而多值依赖X→→Y若在R(U)上成立，不能断言对于任何Y′⊂ Y有X→→Y′成立。

6.2.9 规范化小结

- 关系数据库的规范化理论是数据库逻辑设计的工具。
- 目的：尽量消除插入、删除异常，修改复杂，数据冗余。
- 基本思想：逐步消除数据依赖中不合适的部分。
- 实质：概念的单一化

关系模式的规范化过程

消除决定因素非码的非平凡函数依赖

- 1NF
  ↓ 消除非主属性对码的部分函数依赖
- 2NF
  ↓ 消除非主属性对码的传递函数依赖
- 3NF
  ↓ 消除主属性对码的部分和传递函数依赖
- BCNF
  ↓ 消除非平凡且非函数依赖的多值依赖
- 4NF

- 不能说规范化程度越高的关系模式越好
- 在设计数据库模式结构时，必须对现实世界的实际情况和用户应用需求作进一步分析，确定一个合适的、能够反映现实世界的模式。
- 规范化步骤可以在其中任何一步终止

Chapter 7 数据库设计

数据库设计过程的6个阶段：Page 209

① 需求分析：数据字典

② 概念结构设计：关键阶段，概念模型

③ 逻辑结构设计：关系数据库设计

④ 物理结构设计：存储结构与存取方法

⑤ 数据库实施

⑥ 数据库运行和维护

E-R图向关系模型转换要解决的问题

① 如何将实体型和实体间的联系转换为关系模式？

② 如何确定这些关系模式的属性和码？

转换内容

– 将E-R图转换为关系模型：将实体、实体的属性和实体之间的联系转换为关系模式

实体型间的联系有以下不同情况

① 一个1:1联系可以转换为一个独立的关系模式，也可以与任意一端对应的关系模式合并

② 一个1:n联系可以转换为一个独立的关系模式，也可以与n端对应的关系模式合并

③ 一个m:n联系转换为一个关系模式

Chapter 8 数据库编程

- 嵌入式SQL是将SQL语句嵌入程序设计语言中，被嵌入的程序设计语言，如C、C++、Java，称为宿主语言，简称主语言。

游标

- 游标是系统为用户开设的一个数据缓冲区，存放SQL语句的执行结果。
- 每个游标区都有一个名字
- 用户可以用SQL语句逐一从游标中获取记录，并赋给主变量，交由主语言进一步处理。

Chapter 9 关系查询处理和查询优化

连接操作的实现

- 嵌套循环方法(nested loop)
- 排序-合并方法(sort-merge join 或 merge join)
- 索引连接(index join)方法
- 哈希连接(Hash Join)方法

物理优化选择的方法

- 基于规则的启发式优化
  - 启发式规则是指那些在大多数情况下都适用，但不是在每种情况下都适用的规则。
- 基于代价估算的优化
  - 优化器估算不同执行策略的代价，并选出具有最小代价的执行计划。

二、物理优化连接操作启发式规则

1. 如果两个表都已按照连接属性排序，选用排序—合并方法
2. 如果一个表在连接属性上有索引，选用索引连接方法
3. 如果上面规则都不适用，其中一个表较小，则选用Hash join方法
4. 可以选用嵌套循环方法，并选择其中较小的表，确切地讲是占用的块数较少的表，作为外表(外循环的表)

Chapter 10 数据库恢复技术

事务(Transaction)的基本概念

所谓事务是用户定义的一个数据库操作序列，这些操作要么全做要么全不做，是一个不可分割的工作单位。

事务的特性(ACID)

① 原子性 (Atomicity)
② 一致性 (Consistency)
③ 隔离性 (Isolation)
④ 持续性 (Durability)

ACID每个特性都要理解。
恢复技术能保证事务的原子性、持续性。
并发控制技术能保证事务的一致性，隔离性。

为什么要先写日志文件?

① 写数据库和写日志文件是两个不同操作

② 在这两个操作之间可能发生故障

③ 如果先写了数据库修改，而在日志文件中没有登记下这个修改，则以后就无法恢复这个修改了。

④ 如果先写日志，但没有修改数据库，按日志文件恢复时只不过是多执行一次不必要的UNDO操作，并不影响数据库正确性。

Chapter 11 并发控制

问题的产生

- 多用户数据库系统的存在
- 允许多个用户同时使用数据库系统
  - 飞机定票数据库系统
  - 银行数据库系统
  - 学生选课系统
- 特点：在同一时刻并发运行的事务可达数百个

并发操作带来的数据不一致性

① 丢失修改

② 不可重复读

③ 读“脏”数据

理解具体例子 Page 311

死锁

1. 死锁产生的原因：两个或多个事务都已封锁了一些数据对象，然后又都请求对已为其他事务封锁的数据对象加锁，从而出现互相等待。
2. 死锁的预防方法：一次封锁法、顺序封锁法
3. 死锁的诊断方法：超时法、等待图法